

Web Application Security Testing Report

Target Application: DVWA / Pentest-Ground

Tools Used: Burp Suite and Kali Linux

1. Introduction

This report documents the web application security testing performed on a controlled DVWA and Pentest-Ground environment. The purpose of this testing was to identify common web application vulnerabilities and understand how insecure input handling can affect an application.

The testing focused mainly on Command Injection and Reflected Cross-Site Scripting (XSS). Burp Suite was used to inspect HTTP requests and responses, while Kali Linux was used as the testing environment.

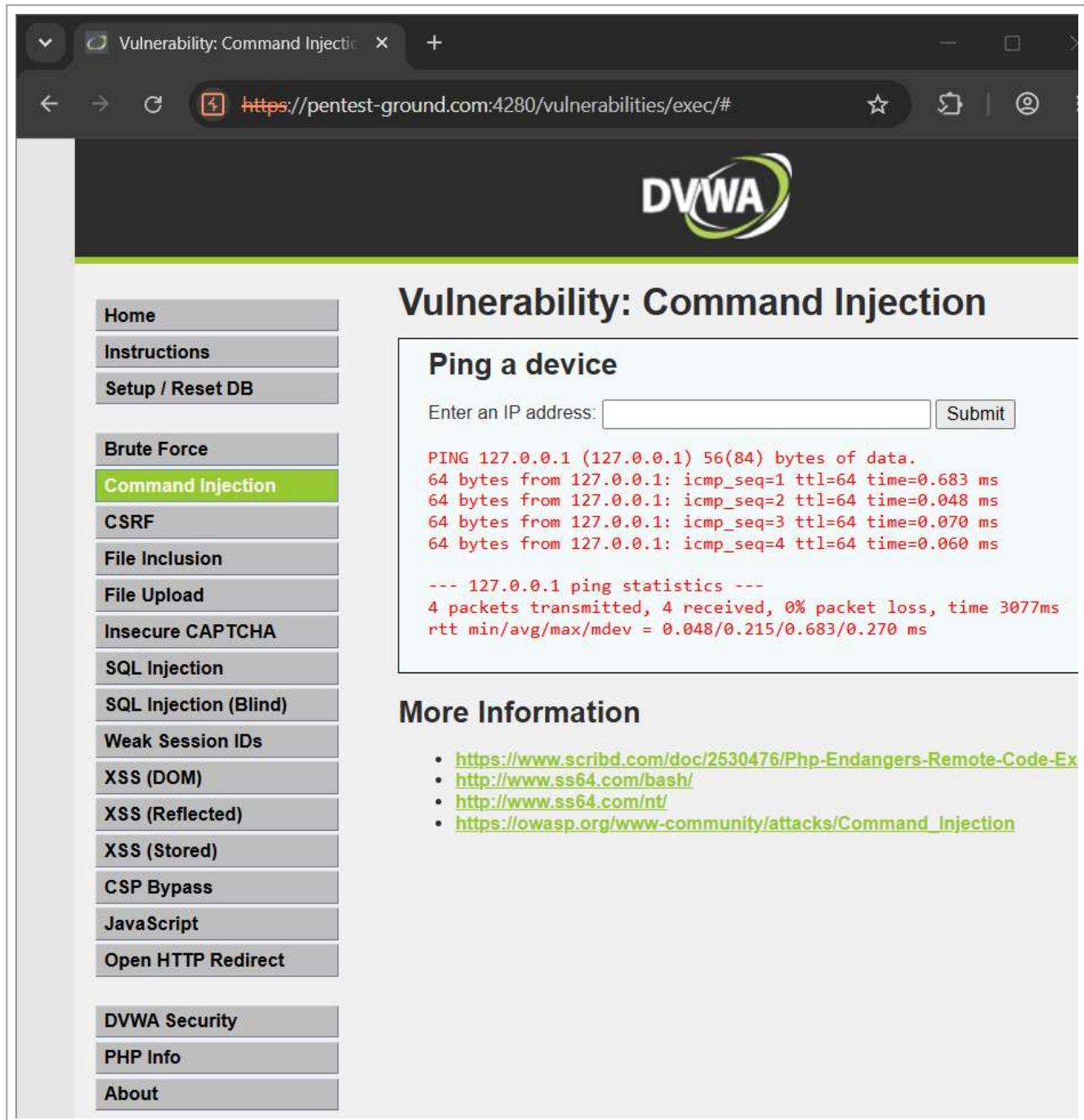
2. Tools Used

- Kali Linux
- Burp Suite Community Edition
- DVWA
- Web Browser

3. Command Injection Testing

Command Injection occurs when an application passes user-controlled input to a system command without proper validation. This can allow unintended commands to be executed by the underlying operating system. Testing was performed only in the authorized DVWA laboratory environment.

Normal Application Test



The screenshot shows a web browser window with the URL `https://pentest-ground.com:4280/vulnerabilities/exec/#`. The page title is "Vulnerability: Command Injection". The DVWA logo is visible at the top. On the left, a sidebar contains a list of vulnerability categories, with "Command Injection" highlighted in green. The main content area is titled "Vulnerability: Command Injection" and features a section "Ping a device". This section includes a form with the label "Enter an IP address:" and a "Submit" button. Below the form, the output of a ping command is displayed in red text: `PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data. 64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.683 ms 64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.048 ms 64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.070 ms 64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.060 ms`. Below this, the ping statistics are shown: `--- 127.0.0.1 ping statistics --- 4 packets transmitted, 4 received, 0% packet loss, time 3077ms rtt min/avg/max/mdev = 0.048/0.215/0.683/0.270 ms`. At the bottom, a "More Information" section lists four links: <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Ex>, <http://www.ss64.com/bash/>, <http://www.ss64.com/nt/>, and https://owasp.org/www-community/attacks/Command_Injection.

Figure 1: Command Injection Normal Test

Proof of Concept

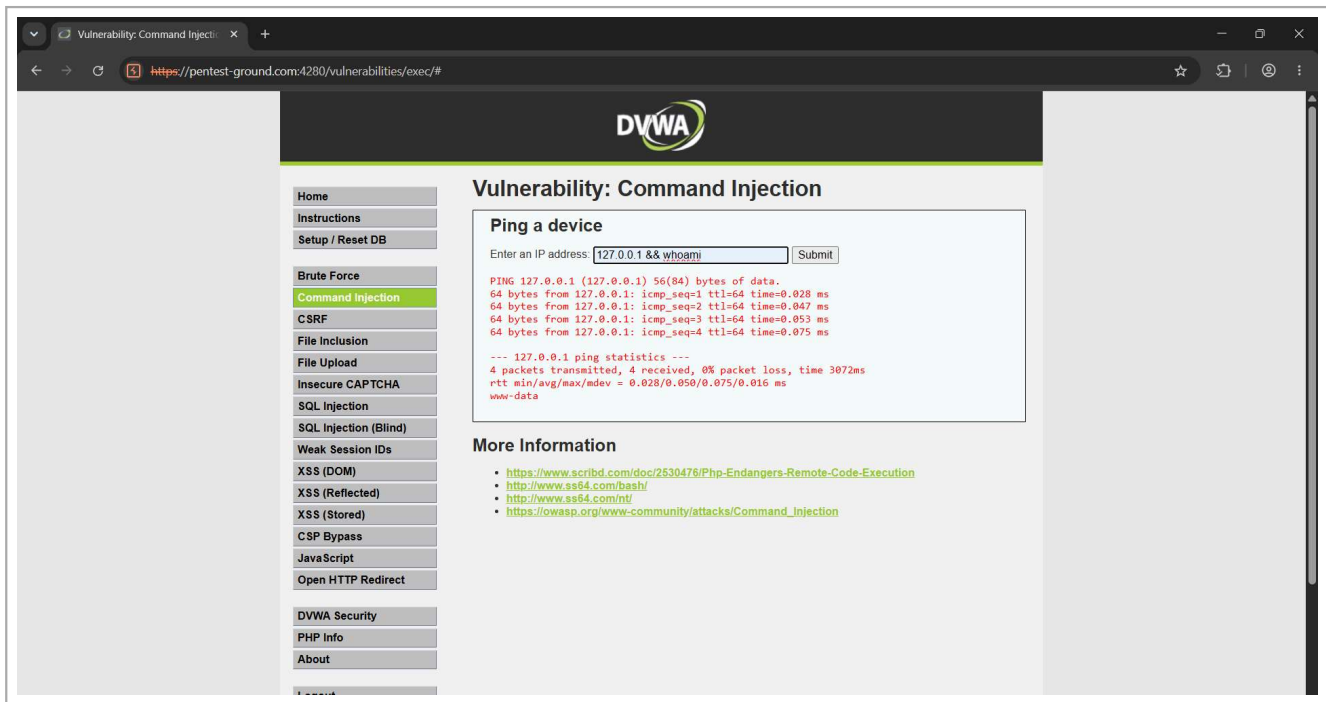


Figure 2: Command Injection Proof of Concept

The result demonstrated that the application accepted input that could influence the system command being executed. This shows the importance of validating and sanitizing user input before passing it to system commands.

4. Environment Setup

Before performing the vulnerability testing, the DVWA application was accessed and its security configuration was verified. The following screenshots show the login page, setup check, and low security configuration used for testing.

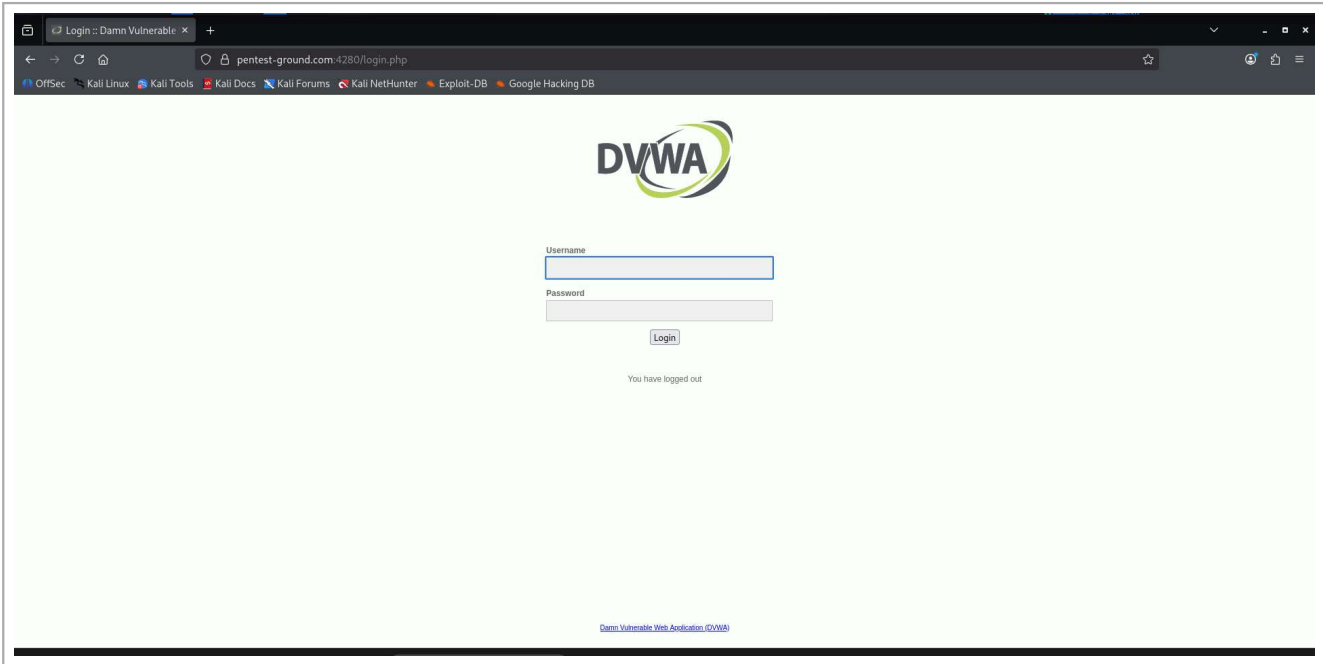


Figure 3: DVWA Login Page

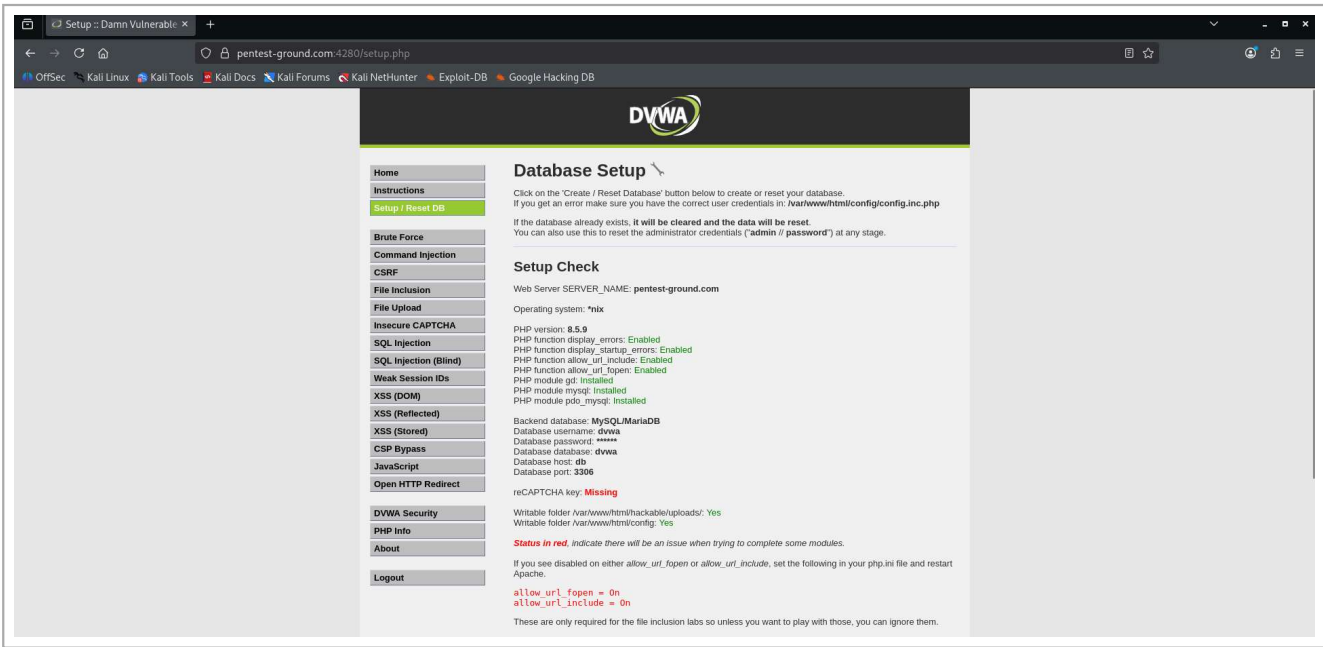


Figure 4: DVWA Setup Check

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Open HTTP Redirect

DVWA Security

PHP Info

About

Logout

Security Level

Security level is currently: **low**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Low

Figure 5: DVWA Security Configuration

5. Burp Suite Request Interception

Burp Suite was configured as a proxy to intercept and inspect HTTP requests sent between the browser and the target application. This helped in understanding how user input was transmitted to the server.

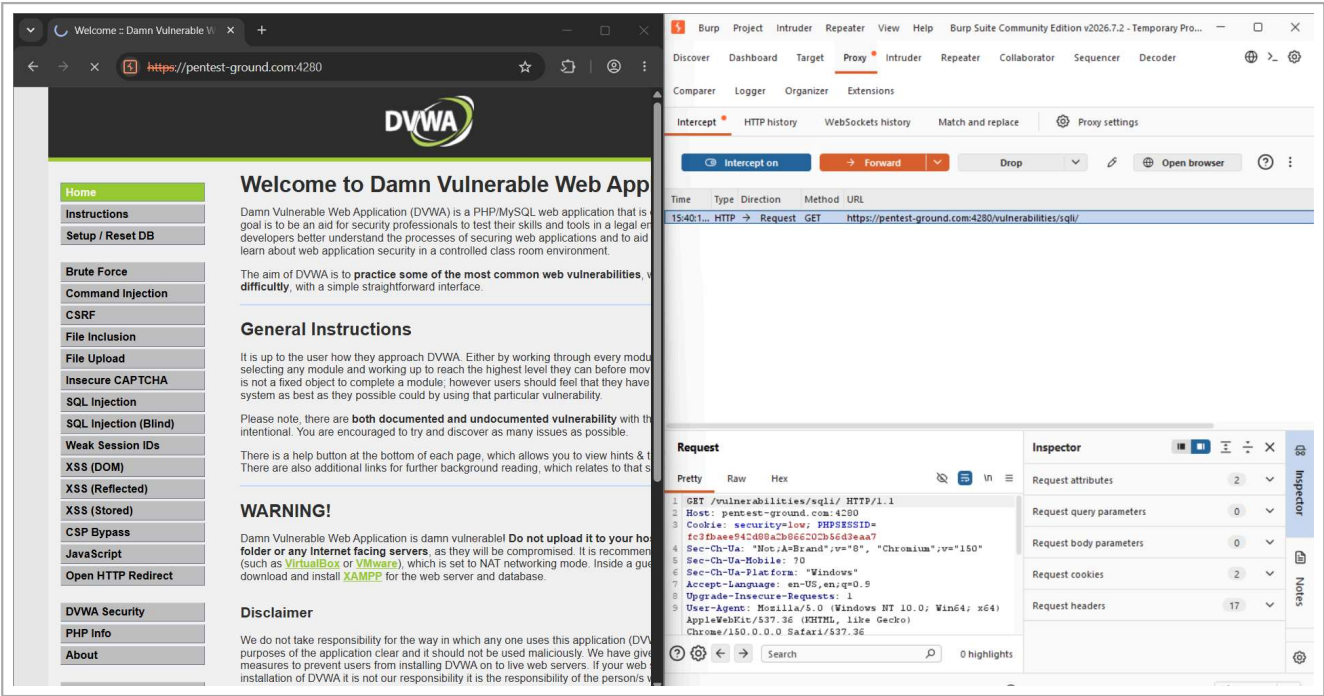


Figure 6: HTTP Request Intercepted Using Burp Suite

6. Reflected Cross-Site Scripting Testing

Reflected Cross-Site Scripting occurs when user-supplied input is immediately returned by the web application without proper output encoding or validation. This can result in client-side code being executed in the victim's browser.

Normal Application Behaviour

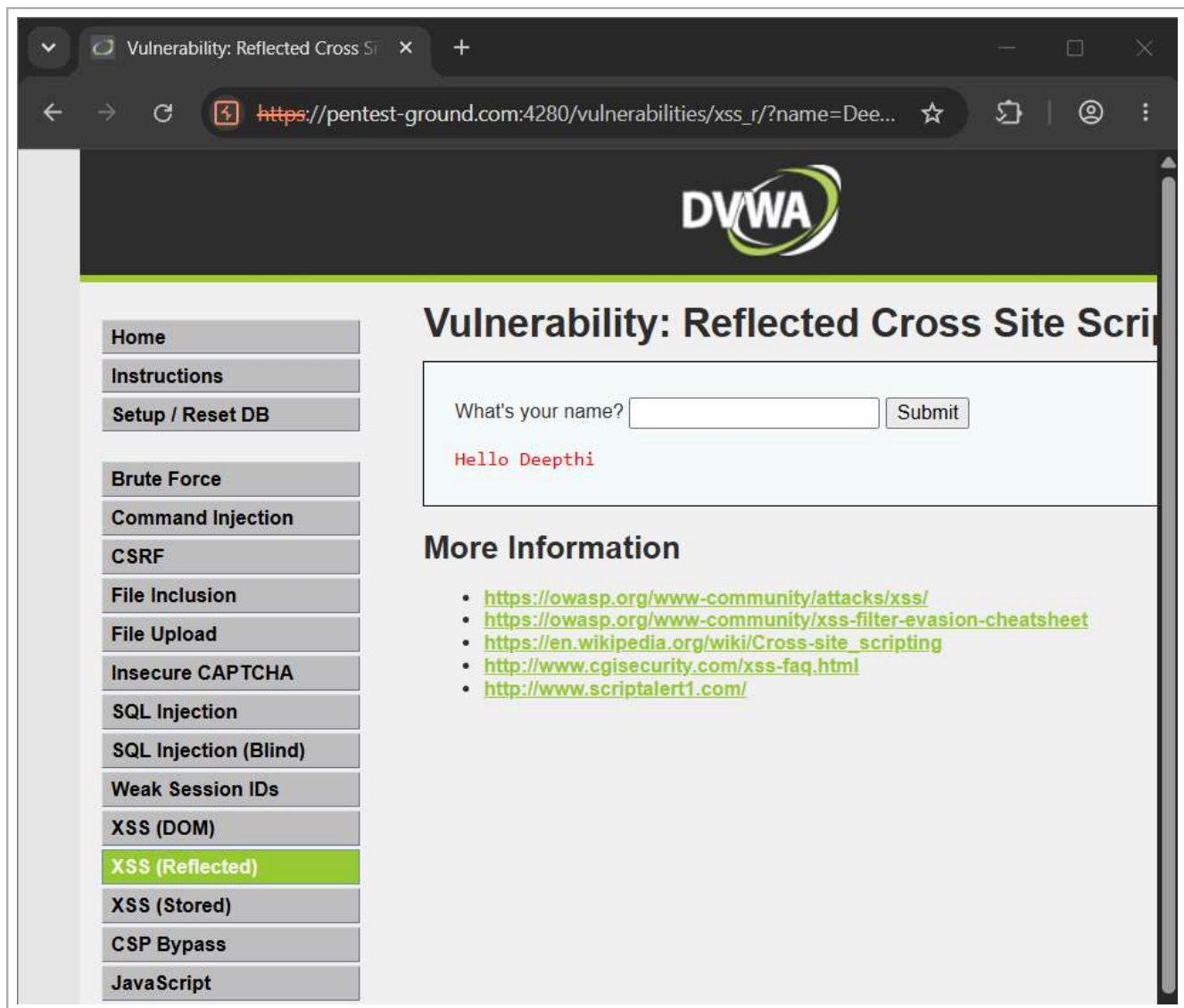


Figure 7: Normal Reflected XSS Application Behaviour

Reflected XSS Proof of Concept

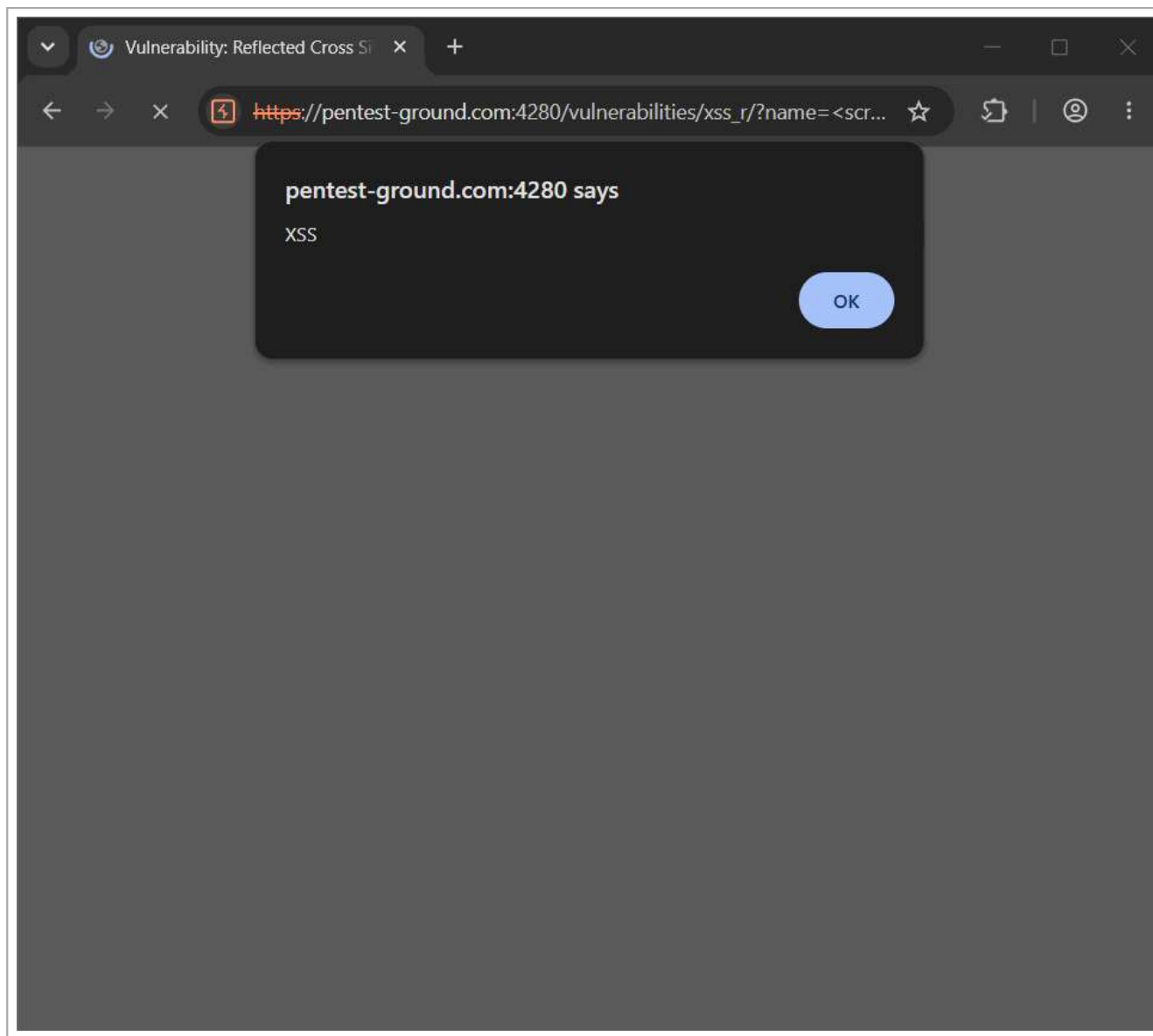


Figure 8: Reflected XSS Proof of Concept

The testing confirmed that reflected user input could be interpreted by the browser, demonstrating the presence of a Reflected XSS vulnerability in the controlled testing environment.

7. SQL Injection Testing

SQL Injection testing was attempted in the DVWA environment. However, the database configuration was not functioning correctly and the required database tables were missing. Because of this configuration issue, SQL Injection testing could not be completed successfully.

The application displayed database-related errors indicating that the required DVWA database tables did not exist. Therefore, SQL Injection was documented as an attempted test that could not be completed due to the environment configuration problem.

8. Findings Summary

Finding 1: Command Injection

User input was able to influence command execution in the intentionally vulnerable laboratory environment.

Finding 2: Reflected Cross-Site Scripting

User-controlled input was reflected in the application response, demonstrating the risk of client-side script execution.

Finding 3: SQL Injection

SQL Injection testing could not be completed because the DVWA database was not properly configured and required tables were missing.

9. Recommendations

- Validate all user input using allowlists where possible.
- Avoid directly passing user input to operating system commands.
- Use secure APIs instead of constructing system commands manually.
- Encode output before displaying user-controlled data in web pages.
- Use parameterized queries to prevent SQL Injection.
- Do not display detailed database or server errors to users.
- Regularly perform security testing and vulnerability assessments.

10. Conclusion

During this web application security testing, Command Injection and Reflected Cross-Site Scripting vulnerabilities were successfully demonstrated in an authorized and controlled testing environment. Burp Suite was used to inspect HTTP requests, while Kali Linux was used as the testing platform.

SQL Injection testing could not be completed because of a database configuration error. Overall, the testing demonstrated the importance of proper input validation, output encoding, secure database handling, and regular security testing of web applications.

Web Application Security Testing Report

DVWA | Burp Suite | Kali Linux

GitHub Repository:

https://github.com/Deepthi452005/NovaShyld-Task_4